

Inteligência Artificial e Aprendizagem de Máquinas

ATIVIDADE P3

Previsão da nota final de alunos com Machine Learning a partir de horas de estudo, frequência, atividades entregues e nota anterior

Desirée Constantino de Almeida Barboza
Nicole Milanez Oliveira

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO	3
2 METODOLOGIA	3
3 DESENVOLVIMENTO DO MODELO	3
4 RESULTADOS	4
5 ANÁLISE DOS RESULTADOS	6
REFERÊNCIAS	7
APÊNDICE A - CÓDIGO FONTE DO MODELO	8

1 INTRODUÇÃO

Esta atividade tem como objetivo criar uma base de dados fictícia de alunos e usar essa base para treinar um modelo de machine learning capaz de prever a nota final de cada aluno. As informações usadas para fazer essa previsão foram as horas de estudo por semana, a frequência nas aulas, a quantidade de atividades entregues e a nota que o aluno tirou anteriormente.

Depois de treinar o modelo, foram calculadas algumas estatísticas para entender se as previsões estavam próximas da realidade, como a média, o desvio padrão e o intervalo de erro. Ao final, esses resultados foram interpretados para avaliar se o desempenho do modelo pode ser considerado bom.

2 METODOLOGIA

A base de dados foi criada de forma artificial, com o auxílio da biblioteca numpy, que gera números aleatórios dentro de intervalos definidos. Foram criados dados de 80 alunos falsos.

As quatro características usadas para prever a nota final foram as horas de estudo por semana, entre 1 e 15 horas, a frequência nas aulas, entre 50% e 100%, a quantidade de atividades entregues, entre 0 e 10, e a nota anterior do aluno, entre 3 e 10.

A nota final de cada aluno foi calculada a partir de uma combinação dessas quatro, com pesos diferentes para cada uma, e adicionando de um valor aleatório. Porque faz com que o resultado não seja certinho e o modelo precise realmente aprender o padrão dos dados, assim como aconteceria com dados verdadeiros.

Para a criação do modelo foi usada a biblioteca scikit-learn, com o algoritmo de regressão linear. A base de dados foi separada em duas partes, sendo 75% para treinar o modelo e 25% para testar se as previsões estavam corretas.

3 DESENVOLVIMENTO DO MODELO

Depois de treinado com os dados de treino, o modelo foi usado para prever a nota dos alunos que ficaram separados para teste e as previsões foram comparadas com as notas reais desses alunos para saber o tamanho do erro em cada caso.

Amostra dos alunos fictícios criados na base de dados.

Aluno	Horas estudo	Frequência	Ativ. entregues	Nota anterior	Nota final
1	6,24	93,16	7	6,46	5,81
2	14,31	81,16	4	4,25	5,67
3	11,25	66,54	3	5,57	4,38
4	9,38	53,18	1	8,21	4,41
5	3,18	65,55	5	8,05	4,28
6	1,81	86,48	10	6,80	6,59

Tabela 1 - Amostra da base de dados de alunos

4 RESULTADOS

Os cálculos das estatísticas com base nas previsões feitas para os alunos do grupo de teste:

Métrica	Valor
Média das notas previstas	5,15
Desvio padrão das previsões	1,44
Erro médio absoluto (MAE)	0,39
Raiz do erro quadrático médio (RMSE)	0,49
Coefficiente de determinação (R^2)	0,87
Média dos erros	-0,13
Desvio padrão dos erros	0,48
Intervalo de erro (95% de confiança)	-1,07 a 0,81

Tabela 2 - Estatísticas do modelo

Essa parte dos cálculos usa funções prontas do Python que fazem a média e o desvio padrão automaticamente, mas a fórmula usada é a mesma da estatística normal.

`np.mean()` calcula a média, `np.std()` calcula o desvio padrão, `mean_absolute_error()` calcula o erro médio absoluto (MAE), `mean_squared_error()` junto com `np.sqrt()` calcula o erro quadrático médio e a raiz dele (RMSE) e `r2_score()` calcula o coeficiente de determinação (R^2)

$\text{media_erro} - 1.96 * \text{desvio_erro}$ e $\text{media_erro} + 1.96 * \text{desvio_erro}$

O gráfico mostra a relação entre as horas de estudo e a nota final de todos os alunos da base de dados. Apesar de ter uma tendência de notas mais altas quando o aluno estuda mais horas, essa relação não é certinha, porque a nota final depende de várias características e não só do tempo de estudo.

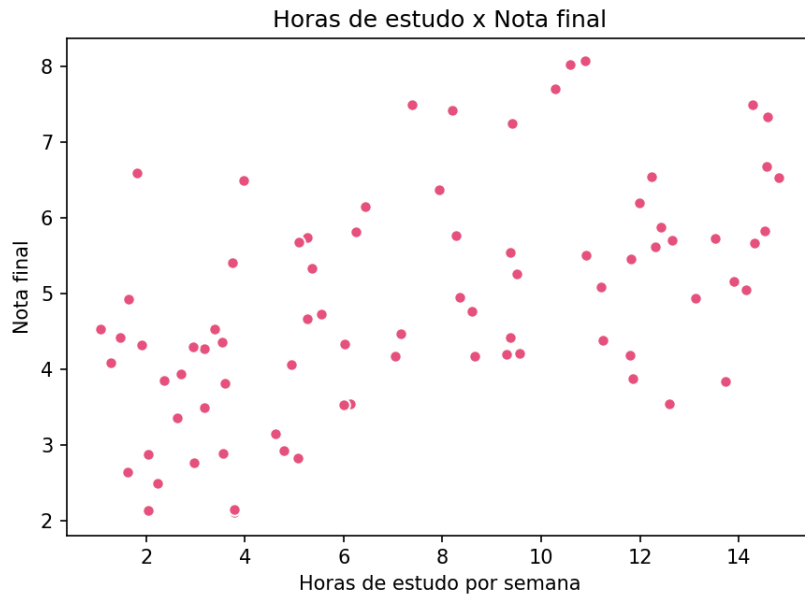


Gráfico 1 - Horas de estudo x Nota final

O gráfico abaixo compara a nota real de cada aluno do grupo de teste com a nota que o modelo previu para ele. Quanto mais perto os pontos estão da linha, melhor foi a previsão do modelo.

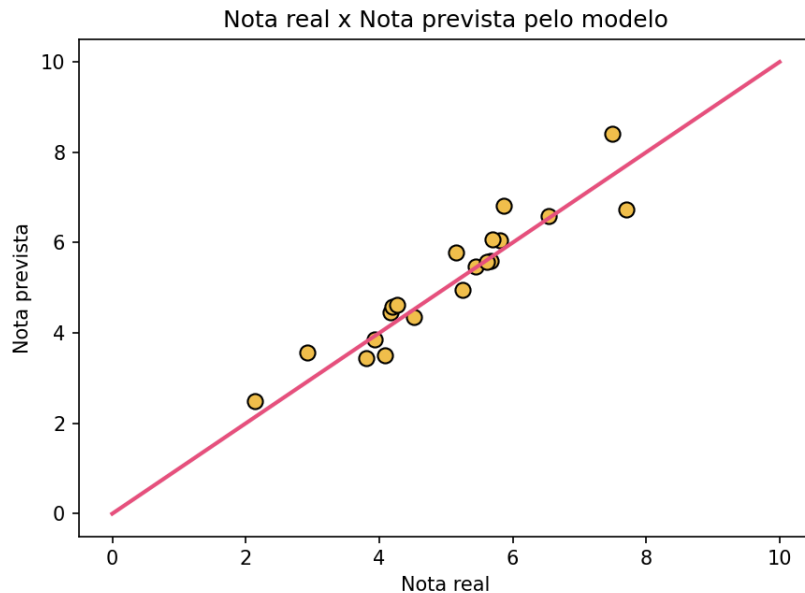


Gráfico 2 - Nota real x Nota prevista pelo modelo

O gráfico abaixo mostra a distribuição dos erros das previsões, então a diferença entre a nota real e a nota prevista em cada aluno do grupo de teste e a linha amarela marca a média desses erros.

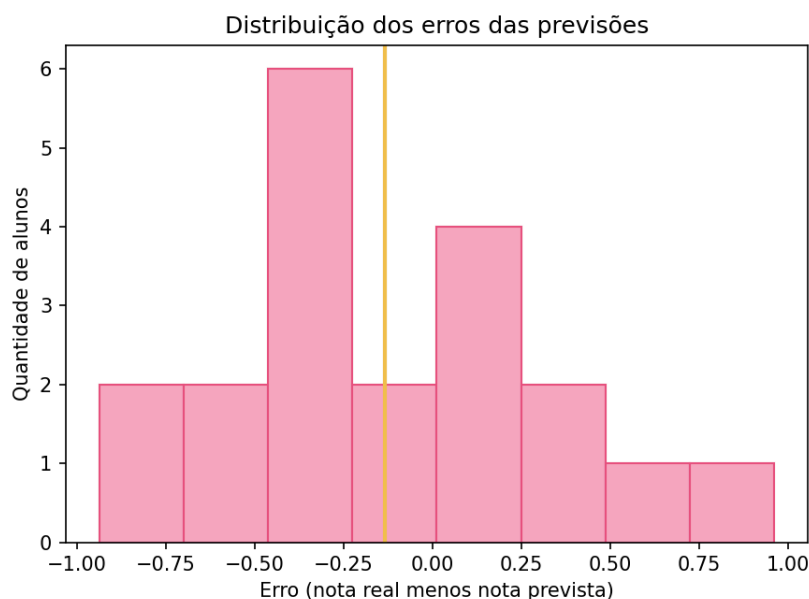


Gráfico 3 - Distribuição dos erros das previsões

Por fim, o gráfico abaixo mostra o peso que o modelo deu para cada característica na hora de calcular a nota final e quanto maior a barra, mais aquela informação influenciou o resultado da previsão.

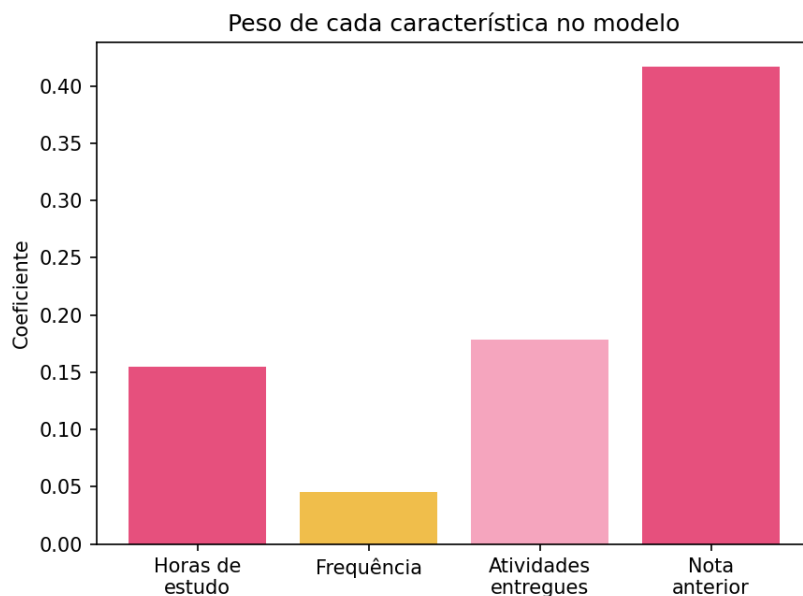


Gráfico 4 - Peso de cada característica no modelo

5 ANÁLISE DOS RESULTADOS

O modelo mostrou um bom desempenho ao prever as notas finais em uma escala de 0 a 10. Ele cometeu um erro médio absoluto muito pequeno, de só 0,39 ponto uns 4 décimos, mantendo uma margem de erro segura entre -1,07 e 0,81 com 95% de certeza. Seu coeficiente

de determinação (R^2) de 0,87 confirma que o algoritmo consegue explicar 87% da variação das notas com base nas quatro informações utilizadas. Entre essas variáveis, a nota anterior foi a de maior impacto no resultado, e as atividades entregues, horas de estudo e, em menor escala, foi a frequência. Mas de qualquer forma por ele apresentar erros baixos e um R^2 alto, o modelo de regressão linear teve um resultado bom para uma base de dados que não é real.

REFERÊNCIAS

SCIKIT-LEARN. Linear Regression. Documentação oficial do scikit-learn. Disponível em: https://scikit-learn.org/stable/modules/linear_model.html.

NUMPY. Documentação oficial do NumPy. Disponível em: <https://numpy.org/doc/>.

MATPLOTLIB. Documentação oficial do Matplotlib. Disponível em: <https://matplotlib.org/stable/index.html>.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação, trabalhos acadêmicos, apresentação. Rio de Janeiro: ABNT, 2011.

APÊNDICE A - CÓDIGO FONTE DO MODELO

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

np.random.seed(42)
n = 80

# base de alunos
horas_estudo = np.random.uniform(1, 15, n)
frequencia = np.random.uniform(50, 100, n)
atividades_entregues = np.random.randint(0, 11, n)
nota_anterior = np.random.uniform(3, 10, n)

ruído = np.random.normal(0, 0.5, n)
nota_final = np.clip(10 * (
    0.25 * (horas_estudo / 15)
    + 0.25 * ((frequencia - 50) / 50)
    + 0.20 * (atividades_entregues / 10)
    + 0.30 * ((nota_anterior - 3) / 7)
) + ruído, 0, 10)

dados = pd.DataFrame({
    "horas_estudo": horas_estudo, "frequencia": frequencia,
    "atividades_entregues": atividades_entregues,
    "nota_anterior": nota_anterior, "nota_final": nota_final
})
dados.to_csv("base_alunos.csv", index=False)

# modelo de regressao linear
x = dados[["horas_estudo", "frequencia", "atividades_entregues",
"nota_anterior"]]
y = dados["nota_final"]
x_treino, x_teste, y_treino, y_teste = train_test_split(x, y,
test_size=0.25, random_state=42)

modelo = LinearRegression().fit(x_treino, y_treino)
```

```

previsoes = np.clip(modelo.predict(x_teste), 0, 10)
erros = y_teste.values - previsoes

# calculos
print("media das previsoes:", round(np.mean(previsoes), 2))
print("desvio padrao das previsoes:", round(np.std(previsoes, ddof=1),
2))
print("mae:", round(mean_absolute_error(y_teste, previsoes), 2))
print("rmse:", round(np.sqrt(mean_squared_error(y_teste, previsoes)),
2))
print("r2:", round(r2_score(y_teste, previsoes), 2))
media_erro, desvio_erro = np.mean(erros), np.std(erros, ddof=1)
print("media do erro:", round(media_erro, 2))
print("desvio padrao do erro:", round(desvio_erro, 2))
print("intervalo de erro 95%:", round(media_erro - 1.96 * desvio_erro,
2), "a", round(media_erro + 1.96 * desvio_erro, 2))

rosa, amarelo, rosa_claro = "#e75480", "#f2c14e", "#f6a5c0"

def salvar_grafico(nome):
    plt.tight_layout()
    plt.savefig(nome, dpi=150)
    plt.close()

plt.figure(figsize=(6, 4.5))
plt.scatter(dados["horas_estudo"], dados["nota_final"], color=rosa,
edgecolor="white")
plt.title("Horas de estudo x Nota final")
plt.xlabel("Horas de estudo por semana")
plt.ylabel("Nota final")
salvar_grafico("grafico_horas_notas.png")

plt.figure(figsize=(6, 4.5))
plt.scatter(y_teste, previsoes, color=amarelo, edgecolor="black", s=60)
plt.plot([0, 10], [0, 10], color=rosa, linewidth=2)
plt.title("Nota real x Nota prevista pelo modelo")
plt.xlabel("Nota real")
plt.ylabel("Nota prevista")
salvar_grafico("grafico_real_previsto.png")

plt.figure(figsize=(6, 4.5))
plt.hist(erros, bins=8, color=rosa_claro, edgecolor=rosa)

```

```

plt.axvline(media_erro, color=amarelo, linewidth=2)
plt.title("Distribuição dos erros das previsões")
plt.xlabel("Erro (nota real menos nota prevista)")
plt.ylabel("Quantidade de alunos")
salvar_grafico("grafico_erros.png")

plt.figure(figsize=(6, 4.5))
nomes = ["Horas de\nestudo", "Frequência", "Atividades\nentregues",
"Nota\anterior"]
plt.bar(nomes, modelo.coef_, color=[rosa, amarelo, rosa_claro, rosa])
plt.title("Peso de cada característica no modelo")
plt.ylabel("Coeficiente")
salvar_grafico("grafico_coeficientes.png")

print("graficos e base de dados gerados")

```

The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a folder named 'Atividade_p3' containing several files: 'base_alunos.csv', 'grafico_coeficientes.png', 'grafico_erros.png', 'grafico_horas_notas.png', 'grafico_real_previsto.png', and 'ModeloNotas.py'. The 'ModeloNotas.py' file is open in the editor, showing the following code:

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.linear_model import LinearRegression
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
7
8 np.random.seed(42)
9 n = 80
10
11 # base de alunos
12 horas_estudo = np.random.uniform(1, 15, n)
13 frequencia = np.random.uniform(50, 100, n)
14 atividades_entregues = np.random.randint(0, 11, n)

```

The terminal output shows the execution of the script, displaying various statistics and a success message:

```

PS C:\Users\Desirée Barboza\Downloads\Atividade_p3> & "C:\Users\Desirée Barboza\AppData\Local\Python\python-core-3.14-64\python.exe" "c:/Users/Desirée Barboza/Downloads/Atividade_p3/ModeloNotas.py"
media das previsões: 5.15
desvio padrao das previsões: 1.44
mae: 0.39
rmse: 0.49
r2: 0.87
media do erro: -0.13
desvio padrao do erro: 0.48
intervalo de erro 95%: -1.07 a 0.81
graficos e base de dados gerados com sucesso
PS C:\Users\Desirée Barboza\Downloads\Atividade_p3>

```